

Análisis y Diseño de Algoritmos

Tema 1: Algoritmos y Complejidad

Juan Miguel Ortiz de Lazcano Lobato



E.T.S. INGENIERÍA INFORMÁTICA

Algoritmos y Complejidad está bajo licencia
**Creative Commons Reconocimiento-
NoComercial-CompartirIgual 4.0**



Contenido

- Coste computacional de un algoritmo
- Análisis de algoritmos iterativos
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

Contenido

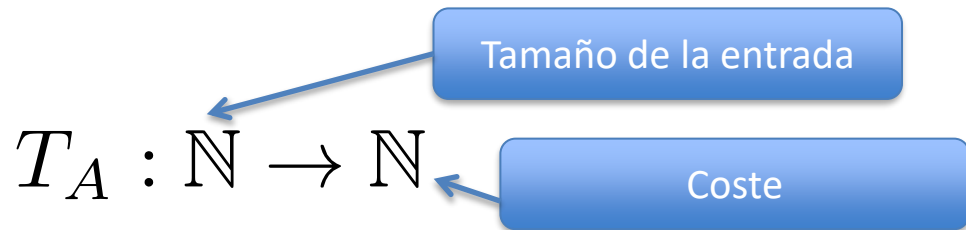
- Coste computacional de un algoritmo
- Análisis de algoritmos iterativos
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

Análisis de Algoritmos

- Parte de la Teoría de la Complejidad que se encarga del estudio del coste en recursos que necesita un algoritmo en el proceso de ejecución.
- Tipos de recursos:
 - **Tiempo**: en general, el coste suele hacer referencia al tiempo empleado en ejecutar el algoritmo.
 - **Espacio**: memoria necesaria para almacenar los datos utilizados por el algoritmo, sin considerar los datos de entrada.
- El factor principal del que depende el coste de un algoritmo es el tamaño de la entrada que tiene que procesar.
 - Normalmente, cuanto mayor es la entrada, más recursos necesita el algoritmo para ejecutarse.
- A veces la naturaleza de los datos de entrada también hay que tenerla en cuenta.
 - Estudiar los casos mejor, peor y medio para una entrada fija de tamaño n .

Complejidad de un algoritmo

- La **complejidad** de un algoritmo A se define como **una función** que asocia a cada entrada del algoritmo su coste en recursos (normalmente tiempo de ejecución)



- Una cuestión importante es determinar el **tamaño** de la **entrada** del algoritmo.
 - Ordenación o búsqueda en una lista: longitud de la lista
 - Evaluación de un polinomio en un punto: grado del polinomio.
 - Multiplicación de matrices cuadradas de tamaño $n \times n$:
 - La dimensión n de la matriz
 - El número de elementos de la matriz $n \times n$
- De forma general podría utilizarse el número de bits de la representación binaria de la entrada.
- A veces la función de la complejidad depende de más de un parámetro de entrada.

Coste Temporal de un Algoritmo

- Se estudiará el coste temporal de las diferentes instrucciones de un lenguaje (Instrucciones simples, composición de instrucciones, sentencias de selección y bucles)
- El tiempo de ejecución en segundos depende de factores que no tienen que ver con el algoritmo como las características del ordenador, el lenguaje de programación utilizado o la carga del sistema en el momento de la ejecución.
- Es necesario una **medida de coste abstracta** e independiente de los factores de bajo nivel.
- Se considera **instrucción simple** aquella con tiempo de ejecución constante y acotado (k segundos). Supondremos que su coste será **1 O.E. (operación elemental)**
- El coste de una sentencia compleja indica el número de operaciones elementales que se han de ejecutar durante la ejecución de dicha sentencia.
 - Hay que analizar el código para ver cómo se comporta dicha instrucción compleja y el coste asociado a su ejecución.

Coste de Instrucciones Simples

- Se consideran instrucciones de coste 1 O.E.:
 - Las expresiones aritméticas, siempre que los datos sean de tamaño constante (+, -, /, %).
 - Las comparaciones de datos simples (<, >, ==, !=) .
 - Las operaciones de asignación (=).
 - La lectura de datos simples desde la entrada estándar (System.in).
 - La escritura de datos simples a la salida estándar (System.out).
 - Las llamadas a procedimientos o funciones (f(...)) .
 - Los saltos en el código (break, return,...)
 - Las operaciones de acceso a:
 - Una componente de un array ([]).
 - Un campo de un registro (struct) o un atributo de un objeto.
 - La siguiente posición de un registro de un archivo en disco duro.

Composición de Instrucciones

Si suponemos que las instrucciones l_1 y l_2 poseen coste computacional (complejidad temporal) de $T_1(n)$ y $T_2(n)$ respectivamente en el caso de estudio, entonces el coste de la composición de las instrucciones será:

$$T_{l_1;l_2}(n) = T_1(n) + T_2(n)$$

```
void swap(int [] a,  
          int i, int j) {  
    int aux = a[i];  
    a[i] = a[j];  
    a[j] = aux;  
}
```

$T_{\text{swap}}(n) =$

```
void swap2(int[] a,  
           int i, int j) {  
    a[i] = a[i] + a[j];  
    a[j] = a[i] - a[j];  
    a[i] = a[i] - a[j];  
}
```

$T_{\text{swap2}}(n) =$

Composición de Instrucciones

Si suponemos que las instrucciones l_1 y l_2 poseen coste computacional (complejidad temporal) de $T_1(n)$ y $T_2(n)$ respectivamente en el caso de estudio, entonces el coste de la composición de las instrucciones será:

$$T_{l_1;l_2}(n) = T_1(n) + T_2(n)$$

```
void swap(int [] a,  
          int i, int j) {  
    int aux = a[i]; --2  
    a[i] = a[j]; --3  
    a[j] = aux; --2  
}
```

$$T_{\text{swap}}(n) = 2 + 3 + 2 = 7$$

```
void swap2(int[] a,  
           int i, int j) {  
    a[i] = a[i] + a[j]; --5  
    a[j] = a[i] - a[j]; --5  
    a[i] = a[i] - a[j]; --5  
}
```

$$T_{\text{swap2}}(n) = 5 + 5 + 5 = 15$$

Coste de las instrucciones de selección

if <condición> then I_1 else I_2

- El coste depende de la evaluación de la condición de control y del coste de la rama que se ejecute (I_1 o I_2)
- La naturaleza de los datos de entrada puede hacer que a veces ejecute I_1 y otras I_2 .
- Normalmente estaremos interesados en estudiar la complejidad en el peor caso.

$$T_{if_else}(n) = T_{condición}(n) + \max\{T_1(n), T_2(n)\}$$

Ejemplo

- Estudia el coste en O.E. del siguiente algoritmo en el peor caso.

```
int max(int a, int b) {  
    int res = b;  
  
    if (a > b) {  
        res = a;  
    }  
    return res;  
}
```

Ejemplo

- Estudia el coste en O.E. del siguiente algoritmo en el peor caso.

```
int max(int a, int b) {  
    int res =b; --1  
  
    if (a > b) { --1  
        res = a; --1  
    }  
    return res; --1  
}
```

- El algoritmo consta de una composición de tres instrucciones:
 1. La asignación de la variable res
 2. La sentencia condicional
 3. La devolución del valor (return)
- El tamaño de la entrada no influye en el coste, pero sí su contenido.
 - El peor caso se dará cuando reciba un valor en a mayor que en b .

$$T_{max}(n) = 1 + (1 + \max(1, 0)) + 1 = 4 \text{ O.E.}$$

Coste de las instrucciones de selección

Case <expresión> of

caso₁: I_1 ;

caso₂: I_2 ;

.....

.....

caso_n: I_n ;

end; {case}

- El coste depende será la suma del coste de ejecutar la expresión de control y el coste de ejecutar la rama correspondiente.
- *El coste de comparar el resultado de la expresión con los valores de los casos se considera 1 O.E., puesto que el número de ramas es finito.*

- En el peor caso: $T_{case}(n) = T_{expresión}(n) + 1 + \max\{T_1(n), \dots, T_n(n)\}$

Coste de los bucles

- Para analizar un bucle hay que entender la semántica del mismo.

```
for (int i= 0; i < K; i++)  
    { bloque con coste C }
```



Inicialización:

```
int i = 0
```

Iteraciones:

```
If (i< K) {bloque coste C; i++}
```

```
If (i< K) {bloque coste C; i++}
```

```
If (i< K) {bloque coste C; i++}
```

...

```
If (i< K) {bloque coste C; i++}
```

Última comprobación de la condición:

```
If (i< K) {bloque coste C; i++}
```

- Se realizan K iteraciones, donde la variable i pasa por los valores $0, 1, 2, \dots, K-1$.
- Cuando $i=K$, la condición de control no es cierta y no se ejecuta el cuerpo del bucle
- Determinar el número de iteraciones que el bucle realiza es fundamental para conocer el coste.

Bucle con Iteraciones Fijas (I)

Si el bucle itera un número fijo de veces K , independiente del tamaño de entrada n , entonces la repetición sólo introduce una constante multiplicativa K .

```
for (int i= 0; i < K; i++)  
{ bloque con coste C }
```

```
int i = 0; while (i < K){  
  {bloque con coste C; i++;}
```

El coste computacional $T(n)$ es $i++ \rightarrow i = i + 1$

$$\begin{aligned} & \text{Coste("int i=0")} + \\ & \left(\sum_{i=0}^{K-1} \text{Coste("i<K")} + C + \text{Coste("i++")} \right) + \\ & \text{Coste("i<K")} = \end{aligned}$$

Inicialización variable control

Coste iteraciones

Última evaluación de $i < K$

$$1 + \left(\sum_{i=0}^{K-1} 1 + 2 + C \right) + 1 = 1 + (3 + C) \cdot K + 1 = 2 + (3 + C) \cdot K = \text{Constante}$$

$$\sum_{i=\min}^{\max} c = c \cdot (\max - \min + 1)$$

Bucle con Iteraciones Fijas (II)

Si el tamaño de la entrada n afecta al número de iteraciones, el coste dependerá de dicho tamaño de entrada.

```
for (int i= 0; i < n; i++)  
{ bloque con coste C }
```

```
int i = 0; while (i < n)  
{bloque con coste C; i++}
```

El bucle realizará **n iteraciones**, tantas como tamaño de los datos de entrada.

$$T(n) = 1 + (1 + 2 + C) \cdot n + 1 = 2 + (3 + C)n$$

Cuando el tamaño de los datos aumenta (n crece), el coste del algoritmo viene determinado prácticamente por el segundo sumando $(3+C) \cdot n$.

$$T(1) = 2 + (3+C) \cdot 1 = 5 + C$$

$$T(10) = 2 + (3+C) \cdot 10 = 2 + (30+10 \cdot C) = 32 + 10 \cdot C$$

$$T(100) = 2 + (3+C) \cdot 100 = 2 + (300+100 \cdot C) = 302 + 100 \cdot C$$

El sumando de la función de complejidad más significativo nos servirá para aproximar la complejidad del algoritmo. En este caso, es $O(n)$.

Bucles Anidados Con Iteraciones Fijas

La variable de control del **bucle interno** es **independiente**.

```
for (int i= 0; i < n; i++) {  
    for (int j= 0; j < n; j++) {  
        bloque con coste C  
    }  
}
```

Cada bucle realiza n iteraciones, tantas como el tamaño de entrada.

$$T(n) = 1 + [1 + 2 + [1 + (1 + 2 + C) \cdot n + 1]] \cdot n + 1 =$$
$$2 + [5 + (3 + C) \cdot n] \cdot n = 2 + 5n + (3 + C)n^2$$

Complejidad cuadrática, $O(n^2)$, dado que el término dependiente de n^2 es el que más hace crecer el coste en la función de complejidad.

$$T(1) = 2 + 5 \cdot 1 + (3 + C) \cdot 1^2 = 2 + 5 + (3 + C) = 10 \cdot C$$

$$T(10) = 2 + 5 \cdot 10 + (3 + C) \cdot 10^2 = 2 + 50 + (300 + 100C) = 352 + 100C$$

$$T(100) = 2 + 5 \cdot 100 + (3 + C) \cdot 100^2 = 2 + 500 + 30000 + 10000C = 30502 + 10000C$$

Bucle Con Iteraciones Variables(I)

El número de iteraciones del bucle interno **depende** de la iteración del bucle externo en la que estamos.

```
for (int i= 0; i < n; i++) {  
    for (int j= 0; j < i; j++) {  
        bloque de coste C  
    }  
}
```

el bucle exterior se realiza n veces, mientras que el interior se realiza 0, 1, 2 ... $n-1$ veces respectivamente.

$$\begin{aligned} T(n) &= 1 + \left[\sum_{i=0}^{n-1} 1 + 2 + \left(1 + \left(\sum_{j=0}^{i-1} 1 + 2 + C \right) + 1 \right) \right] + 1 \\ &= 2 + \left[\sum_{i=0}^{n-1} 5 + (3 + C) \cdot i \right] = 2 + \sum_{i=0}^{n-1} 5 + \sum_{i=0}^{n-1} (3 + C) \cdot i \\ &= 2 + \sum_{i=0}^{n-1} 5 + (3 + C) \sum_{i=0}^{n-1} i = 2 + 5n + (3 + C) \frac{(n-1)(n)}{2} \in \mathbf{O(n^2)} \end{aligned}$$

$\sum_{i=a}^b s_i = \frac{(s_a + s_b)}{2} (b - a + 1)$

Bucle con Iteraciones Variables (II)

A veces aparecen bucles multiplicativos, donde la **evolución de la variable** de control **no** es **lineal** (como en los casos anteriores)

```
c= 1;
while (c < n) {
    bloque de coste A
    c= 2*c;
}
```

Tras Iteración i	c
0 (antes de entrar)	$1 = 2^0$
1	$2 = 1 * 2 = 2^1$
2	$4 = 2 * 2 = 2^2$
3	$8 = 2 * 2 * 2 = 2^3$
...	...
k	2^k

El valor inicial de "c" es 1, siendo 2^k al cabo de k iteraciones.

Si el bucle acaba tras la iteración k, se debe a que la condición de control fue evaluada a falso y se cumple que $c \geq n$. Por ello,

$$2^k \geq n \Rightarrow k \geq \log_2(n) \Rightarrow k = \lceil \log_2(n) \rceil$$

$$T(n) = 1 + (1 + A + 2) \cdot \lceil \log_2(n) \rceil + 1 \in \mathbf{O(\log_2(n))}$$

Coste de los subprogramas

- El coste de una llamada a un subprograma cuya complejidad es $T_{\text{sub}}(n)$ será $1 + T_{\text{sub}}(n)$.

```
boolean satisface(int [] a) {  
    boolean hayValido = false;  
  
    int i = 0;  
    while (i < pos && hayValido == false) {  
        hayValido = cumpleCond(a,i);  
        i++;  
    }  
    return hayValido;  
}
```

```
boolean cumpleCond (int [] a,  
                    int i)  
{return (a[i] < 10); }
```

- Tamaño de entrada:** número de elementos del array a. Es decir, $n = a.length$
- El coste de *cumpleCond* es independiente del tamaño de entrada considerado.

$$T_{\text{cumpleCond}}(n) = 3$$

- En el peor caso:

$$T_{\text{satisface}}(n) = 1 + 1 + \left(\sum_{i=0}^{a.length-1} (3 + (2 + T_{\text{cumpleCond}}(n)) + 2) \right) + 2 + 1 =$$

$$2 + 10n + 3 = 10n + 5$$

Evaluación en cortocircuito.
(i<pos) es falso.

Contenido

- Coste computacional de un algoritmo
- **Análisis de algoritmos iterativos**
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

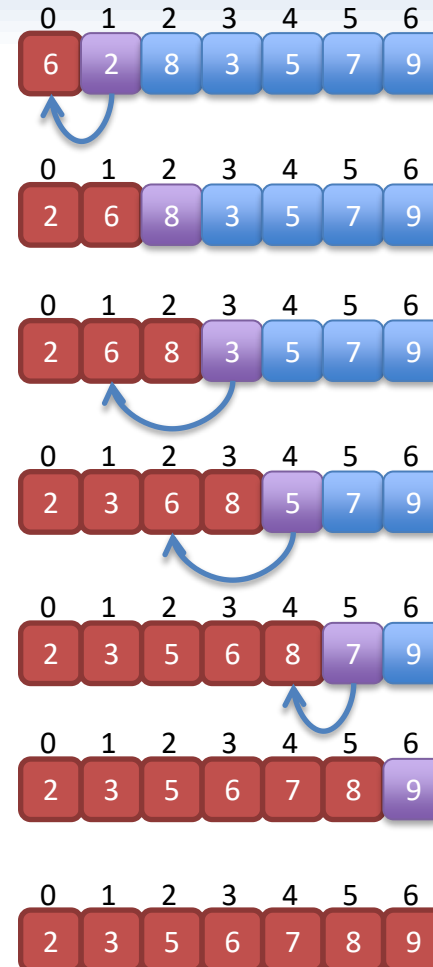
Análisis de algoritmos no recursivos

- Estrategia para analizar la eficiencia en tiempo de algoritmos no recursivos:
 - Decidir cómo medir el tamaño de la entrada
 - Identificar las operaciones básicas que se van a contabilizar (normalmente, las que están en el bucle más interno).
 - Se simplifican las constantes de la expresión resultante.
 - Comprobar si el número de veces que se ejecutan las instrucciones básicas depende sólo del tamaño de la entrada.
 - Si hay otras condiciones a tener en cuenta habrá que estudiar las complejidades en el caso peor, mejor, y medio de manera separada.
 - Establecer la función de complejidad, una suma que defina el número de veces que se ejecutan las operaciones básicas.
 - Aplicar transformaciones matemáticas para que la expresión de la función de complejidad permita establecer el orden de crecimiento (lo veremos más adelante)

Análisis de algoritmos no recursivos:

Ordenación por inserción

```
/**
 *
 * @param a array de enteros a
 * ordenar por inserción
 */
public static void insercion(int[] a){
    for (int i = 1; i < a.length; i++){
        // a[0..i-1] está ordenado
        int j = i-1;
        int x = a[i];
        // insertamos a[i] en a[0..i-1]
        // para que a[0..i] quede ordenado
        while (j >= 0 && a[j] > x) {
            a[j+1] = a[j];
            j--;
        }
        a[j+1] = x;
    }
}
```



Análisis de algoritmos no recursivos:

Ordenación por inserción

```
/**
 *
 * @param a array de enteros a
 * ordenar por inserción
 */
public static void insercion(int[] a){
    for (int i = 1; i<a.length; i++){
        // a[0..i-1] está ordenado
        int j = i-1;
        int x = a[i];
        // insertamos a[i] en a[0..i-1]
        // para que a[0..i] quede ordenado
        while (j>=0 && a[j]>x) {
            a[j+1]= a[j];
            j--;
        }
        a[j+1]=x;
    }
}
```

- **Tamaño de la entrada:** longitud del array a ordenar, $n = a.length$.
- **Operaciones básicas:** acceso a componentes del array.

Análisis de algoritmos no recursivos:

Ordenación por inserción

```
/**
 *
 * @param a array de enteros a
 * ordenar por inserción
 */
public static void insercion(int[] a){
    for (int i = 1; i<a.length; i++){
        // a[0..i-1] está ordenado
        int j = i-1;
        int x = a[i];
        // insertamos a[i] en a[0..i-1]
        // para que a[0..i] quede ordenado
        while (j>=0 && a[j]>x) {
            a[j+1]= a[j];
            j--;
        }
        a[j+1]=x;
    }
}
```

- **Tamaño** de la **entrada**: longitud del array a ordenar, $n = a.length$.
- **Operaciones básicas**: acceso a componentes del array.
- **Caso mejor**: el array está ordenado
 - El bucle anidado no se ejecuta nunca

0	1	2	3	4	5	6
2	3	5	6	7	8	9

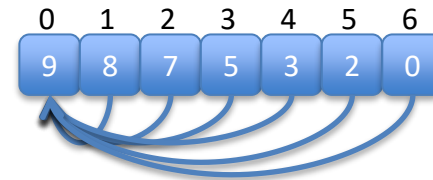
$$T_M(n) = \sum_{i=1}^{n-1} 3 = 3(n-1) = 3n-3 \in O(n)$$

Análisis de algoritmos no recursivos:

Ordenación por inserción

```
/**
 *
 * @param a array de enteros a
 * ordenar por inserción
 */
public static void insercion(int[] a){
    for (int i = 1; i<a.length; i++){
        // a[0..i-1] está ordenado
        int j = i-1;
        int x = a[i];
        // insertamos a[i] en a[0..i-1]
        // para que a[0..i] quede ordenado
        while (j>=0 && a[j]>x) {
            a[j+1]= a[j];
            j--;
        }
        a[j+1]=x;
    }
}
```

- **Caso peor:** el array está ordenado de forma decreciente
 - El bucle anidado se ejecuta siempre



$$T_p(n) = \sum_{i=1}^{n-1} \left[1 + \left(\sum_{j=0}^{i-1} (1 + 2) \right) + 1 \right] =$$

$$\sum_{i=1}^{n-1} (2 + 3 \cdot i) = \sum_{i=1}^{n-1} 2 + 3 \sum_{i=1}^{n-1} i =$$

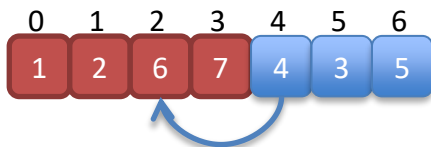
$$2(n-1) + 3 \frac{(1+n-1)}{2} (n-1) =$$

$$2(n-1) + \frac{3}{2} n(n-1) = 1.5n^2 + 0.5n - 2 \in O(n^2)$$

Análisis de algoritmos no recursivos:

Ordenación por inserción

```
/**
 *
 * @param a array de enteros a
 * ordenar por inserción
 */
public static void insercion(int[] a){
    for (int i = 1; i<a.length; i++){
        // a[0..i-1] está ordenado
        int j = i-1;
        int x = a[i];
        // insertamos a[i] en a[0..i-1]
        // para que a[0..i] quede ordenado
        while (j>=0 && a[j]>x) {
            a[j+1]= a[j];
            j--;
        }
        a[j+1]=x;
    }
}
```



- Caso medio:

- Supongamos que escogemos de forma aleatoria n números y aplicamos el algoritmo “insercion”.
- ¿Cuántas iteraciones se realizan para insertar el elemento $a[i]$ en $a[0..i-1]$? De media, la mitad de los números serán menores que $a[i]$, y la otra mitad mayores o iguales que $a[i]$
- El bucle anidado se ejecuta $i/2$ veces

$$\begin{aligned} T_{1/2}(n) &= \sum_{i=1}^{n-1} \left(2 + 3 \frac{i}{2} + 1 \right) = 3(n-1) + \frac{3}{2} \sum_{i=1}^{n-1} i = \\ &= 3(n-1) + \frac{3}{2} \frac{n(n-1)}{2} = \\ &= 3n - 3 + 0.75n^2 - 0.75n = \\ &= 0.75n^2 + 2.25n - 3 \end{aligned}$$

El caso medio es casi tan malo como el caso peor

Contenido

- Coste computacional de un algoritmo
- Análisis de algoritmos iterativos
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

Análisis de algoritmos recursivos

- La estimación de la complejidad de un algoritmo iterativo es en muchas ocasiones directa, gracias al uso de series aritméticas o geométricas.
- En el caso de los algoritmos recursivos, el coste del algoritmo para un determinado tamaño de entrada suele depender del coste del mismo algoritmo para tamaño de entrada inferiores, dando lugar a una ecuación de recurrencia.
- Estrategia:
 - Decidir el parámetro/s que determinan el tamaño de la entrada.
 - Identificar la operación u operaciones básicas del algoritmo.
 - Comprobar si el número de veces que se ejecuta la operación básica depende únicamente del tamaño de la entrada, o hay otras condiciones adicionales que influyen. En este caso, habría que distinguir entre los casos peor, medio y mejor.
 - Establecer la ecuación de recurrencia, con las condiciones iniciales.
 - Resolver la ecuación de recurrencia, o al menos, establecer el orden de crecimiento de su solución (se verá más adelante).

Análisis de Algoritmos Recursivos

Factorial

```
/**
 * @param n número natural
 * @return n! = 1*...*n
 */
public static int factorial(int n){
    if (n<0) throw
        new IllegalArgumentException("número no válido");
    if (n == 0) return 1;
    else return n*factorial(n-1);
}
```

- **Tamaño** de la **entrada**: el valor del número natural
- **Operación básica**: la multiplicación de números
- El cálculo de la complejidad nos lleva de forma natural a una **ecuación de recurrencia**:

$$T(n) = \begin{cases} 0 & n = 0 \\ 1 + T(n-1) & n > 0 \end{cases}$$

Caso base (condiciones iniciales)

Producto de n y factorial(n-1)

Multiplicaciones para calcular factorial(n-1)

Resolución de recurrencias

Sustitución Hacia Atrás

- Hay un gran arsenal matemático disponible para convertir una recurrencia en una ecuación en forma cerrada. El método más simple es la **sustitución hacia atrás**.
- Se calcula la recurrencia para $n-1, n-2, \dots$

$$\begin{aligned}T(n) &= 1 + T(n-1) = 1 + (1 + T(n-1-1)) = 2 + T(n-2) \\&= 2 + (1 + T(n-2-1)) = 3 + T(n-3) = \dots \\&= i + T(n-i)\end{aligned}$$

- La sustitución sigue hasta que se llega al caso base $T(0)$. En ese caso, $0 = n-i \Rightarrow i = n$.
- $T(n) = n + T(0) = n + 0 = n$;

Recurrencias Lineales Homogéneas (I)

- El **método** del **polinomio característico** puede utilizarse cuando la recurrencia tiene la forma

$$T(n) = a_{k-1}T(n - 1) + a_{k-2}T(n - 2) + \cdots + a_0T(n - k)$$

donde cada a_i es una cierta constante. Necesitamos en este caso k condiciones iniciales $T(1), \cdots, T(k)$.

- Se denomina **recurrencia lineal homogénea** de orden k ,

$$1 \cdot T(n) - a_{k-1}T(n - 1) - a_{k-2}T(n - 2) - \cdots - a_1T(n - k + 1) - a_0T(n - k) = 0$$

cuya ecuación característica es $1 \cdot x^k - a_{k-1}x^{k-1} - a_{k-2}x^{k-2} - \cdots - a_1x - a_0 = 0$

- La obtención de las **raíces de la ecuación característica** nos va a permitir resolver la recurrencia.

Recurrencias Lineales Homogéneas (II)

- Si las raíces $\lambda_1, \dots, \lambda_k$ son todas diferentes, la solución a la recurrencia homogénea es

$$T(n) = \alpha_1 \lambda_1^n + \alpha_2 \lambda_2^n + \dots + \alpha_k \lambda_k^n$$

- Los valores de α_i dependen de las condiciones iniciales. Para obtenerlos se resuelve el sistema de ecuaciones que resulta de igualar la ecuación anterior a las condiciones iniciales ($n=1,2,\dots,k$):

$$T(1) = \alpha_1 \lambda_1 + \alpha_2 \lambda_2 + \dots + \alpha_k \lambda_k$$

$$T(2) = \alpha_1 \lambda_1^2 + \alpha_2 \lambda_2^2 + \dots + \alpha_k \lambda_k^2$$

...

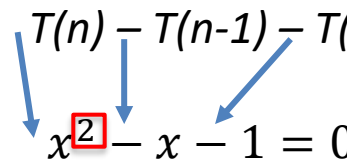
$$T(k) = \alpha_1 \lambda_1^k + \alpha_2 \lambda_2^k + \dots + \alpha_k \lambda_k^k$$

Ejemplo. Resolver Recurrencia Lineal Homogénea

Sucesión de Fibonacci: $T(n) = \begin{cases} 1 & n = 1 \\ 1 & n = 2 \\ T(n-1) + T(n-2) & n > 2 \end{cases}$

La ecuación característica del caso general $T(n) - T(n-1) - T(n-2) = 0$ es

$x^2 - x - 1 = 0$



cuyas raíces son $x = \frac{1 \pm \sqrt{5}}{2}$

La ecuación cerrada será: $T(n) = \alpha_1 \left(\frac{1+\sqrt{5}}{2} \right)^n + \alpha_2 \left(\frac{1-\sqrt{5}}{2} \right)^n$

Sustituyendo los casos iniciales obtenemos el sistema de ecuaciones

$$\begin{aligned} T(1) = 1 &= \alpha_1 \left(\frac{1+\sqrt{5}}{2} \right) + \alpha_2 \left(\frac{1-\sqrt{5}}{2} \right) \\ T(2) = 1 &= \alpha_1 \left(\frac{1+\sqrt{5}}{2} \right)^2 + \alpha_2 \left(\frac{1-\sqrt{5}}{2} \right)^2 \end{aligned}$$

Al resolverlo se obtienen los valores de las constantes: $\alpha_1 = 1/\sqrt{5}$, $\alpha_2 = -1/\sqrt{5}$

Recurrencias Lineales Homogéneas(III)

- Si alguna raíz λ_i es múltiple, el término correspondiente es $p_i(n) \cdot \lambda_i^n$, donde $p_i(n)$ es un polinomio de grado $\mu - 1$, en el que μ es la multiplicidad de la raíz λ_i .
- Ejemplo: Resolver $T(n) = 4T(n - 1) - 4T(n - 2)$, con $T(1) = 1$ y $T(2) = 4$.

La ecuación característica es $x^2 - 4x + 4 = 0$, que tiene raíz doble $x = 2$.

Obtenemos entonces $T(n) = (\alpha_1 + \alpha_2 n) \cdot 2^n$.

Sustituyendo los casos iniciales y resolviendo el sistema de ecuaciones

$$\begin{aligned} T(1) &= 1 = (\alpha_1 + \alpha_2)2 \\ T(2) &= 4 = (\alpha_1 + 2\alpha_2)2^2 \end{aligned}$$

resulta $\alpha_1 = 0$ y $\alpha_2 = 1/2$, por lo que $T(n) = 0.5 n \cdot 2^n$

Recurrencias Lineales No Homogéneas

- La **recurrencia lineal**

$$T(n) = a_{k-1}T(n-1) + a_{k-2}T(n-2) + \cdots + a_0T(n-k) + f(n)$$

es **no homogénea** si $f(n) \neq 0$

- Para ecuaciones de recurrencia lineal no homogéneas del tipo:

$$T(n) = a_{k-1}T(n-1) + a_{k-2}T(n-2) + \cdots + a_0T(n-k) + b^n p(n)$$

con $p(n)$ un polinomio de grado μ , $b \in \mathbb{R}$ y $\forall 1 \leq i \leq k: a_i \in \mathbb{R}$

el polinomio característico que nos permite encontrar la forma de la solución es

$$(x^k - a_{k-1}x^{k-1} - \cdots - a_1x - a_0) (x-b)^{\mu+1} = 0$$

Una vez obtenidas las raíces se procede de la forma descrita para el caso en que la ecuación sea homogénea.

Ejemplo

- Resolver la ecuación $T(n) = 2T(n-1) + 2^n$

$$T(n) - 2T(n-1) = 2^n = 2^n \cdot 1$$

b

$p(n), \mu = 0$

$$(x-2)(x-2)^{0+1} = (x-2)^2 = 0$$

- Es decir, tenemos una única raíz, $x = 2$, con grado de multiplicidad 2.
- Por lo que la solución es de la forma:

$$T(n) = (\alpha_1 + \alpha_2 \cdot n) \cdot 2^n = \alpha_1 \cdot 2^n + \alpha_2 \cdot n \cdot 2^n$$

Recurrencias Lineales No Homogéneas

En general, dada una recurrencia del tipo

$$T(n) = a_{k-1}T(n-1) + \dots + a_0T(n-k) + p_1(n)b_1^n + \dots + p_m(n)b_m^n$$

donde $\forall 1 \leq j \leq m: p_j(n)$ es un polinomio de grado μ_j y $b_j \in \mathbb{R}$, $\forall 1 \leq i \leq k: a_i \in \mathbb{R}$ y $\forall 1 \leq i, j \leq m, i \neq j: b_i \neq b_j$

Entonces el polinomio característico que nos permite encontrar la forma de la solución es

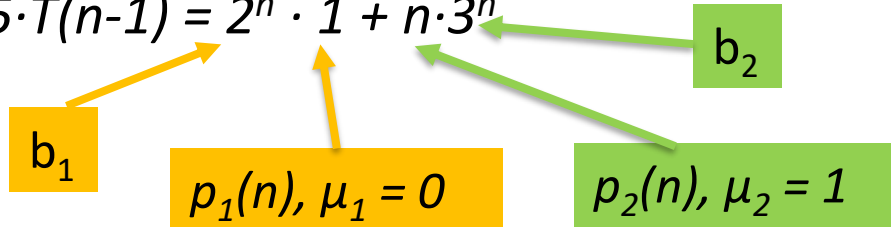
$$(x^k - a_{k-1}x^{k-1} - \dots - a_1x - a_0)(x - b_1)^{\mu_1+1} \cdot \dots \cdot (x - b_m)^{\mu_m+1} = 0$$

Una vez obtenidas las raíces se procede de la misma forma descrita para el caso en que la ecuación sea homogénea.

Ejemplo

- Resolver la ecuación $T(n) = 5 \cdot T(n-1) + 2^n + n \cdot 3^n$

$$T(n) - 5 \cdot T(n-1) = 2^n \cdot 1 + n \cdot 3^n$$



- El polinomio característico es

$$(x-5) (x-2)^{0+1} (x-3)^{1+1} = (x-5) (x-2) (x-3)^2 = 0$$

- Es decir, tenemos dos raíces simples, $x = 5$ y $x = 2$, y una raíz múltiple, $x = 3$, de grado 2.
- Por lo que la solución es de la forma:

$$T(n) = \alpha_1 \cdot 5^n + \alpha_2 \cdot 2^n + \alpha_3 \cdot 3^n + \alpha_4 \cdot n \cdot 3^n$$

Recurrencias No Lineales

- Se aplican transformaciones y cambios de variable para conseguir una recurrencia lineal.

```
/**
 * @param l lista con n > 0 elementos (min <= max)
 * @param min y max, índices mínimo y máximo a sumar
 * @return Suma de los elementos de la lista
 * @Precondición 0 <= min <= max < l.size()
 */
public double suma(List<Double> l, int min, int max) {
    if (max == min) return l.get(min);
    else {
        double s1 = suma(l, min, (min+max)/2);
        double s2 = suma(l, (min+max)/2+1, max);
        return s1+s2;
    }
}
```

- Tamaño** de la **entrada**, n , es el número de elementos de la lista l en el rango $[min, max]$
- Operaciones básicas**: suma de números reales (Double) y accesos a la lista ($l.get()$).
- Complejidad**:

$$T(n) = 2T(n/2) + 1, n > 1$$

$$T(1) = 1$$

- Se hace un cambio de variable. En este caso, $n = 2^k$
$$T(2^k) = 2 T(2^k/2) + 1 = 2 T(2^{k-1}) + 1$$
- Se renombra la función compuesta $T(2^k)$ como $F(k)$

$$F(k) = 2F(k-1) + 1$$

- Se resuelve la recurrencia lineal.

$$F(k) = \alpha_1 2^k + \alpha_2$$

- Hay que deshacer los cambios, en orden inverso al que se aplicaron.

$$T(2^k) = \alpha_1 2^k + \alpha_2;$$

$$T(n) = \alpha_1 \cdot n + \alpha_2;$$

- Calculamos las constantes.

$$T(n) = 2n - 1$$

Contenido

- Coste computacional de un algoritmo
- Análisis de algoritmos iterativos
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

Comparando algoritmos

- Algoritmos para calcular b^n , con $n = 2^i$, $i=1,2,\dots$

- Método directo:

```
//n es potencia de 2
int potencia(int b, int n) {
    int res = b;
    for (int i = 2; i <= n; i++) {
        res = res * b;
    }
    return res;
}
```

- Enfoque alternativo:

```
//n es potencia de 2
int potencia2(int b, int n) {
    int res = b;
    if (n > 1) {
        int aux = potencia2(b, n/2);
        res = aux * aux;
    }
    return res;
}
```

- ¿Cuál es más eficiente?

Comparando Algoritmos

- Complejidad Método Directo:

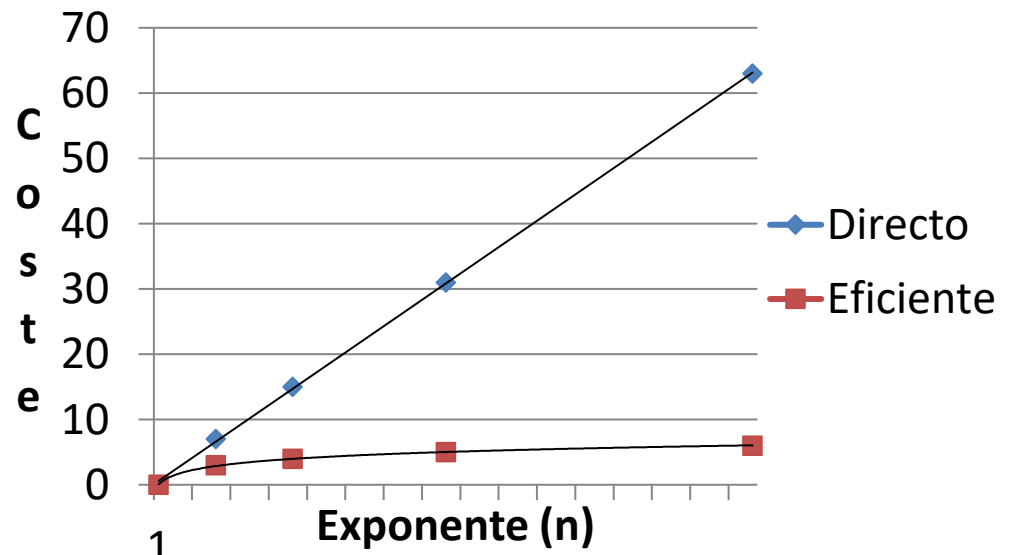
$$T(n) = \sum_{i=2}^n 1 = n - 1$$

- Complejidad Método Alternativo:

$$T(n) = \begin{cases} 0 & n = 1 \\ T\left(\frac{n}{2}\right) + 1 & n > 1 \end{cases}$$

Tras resolver la recurrencia

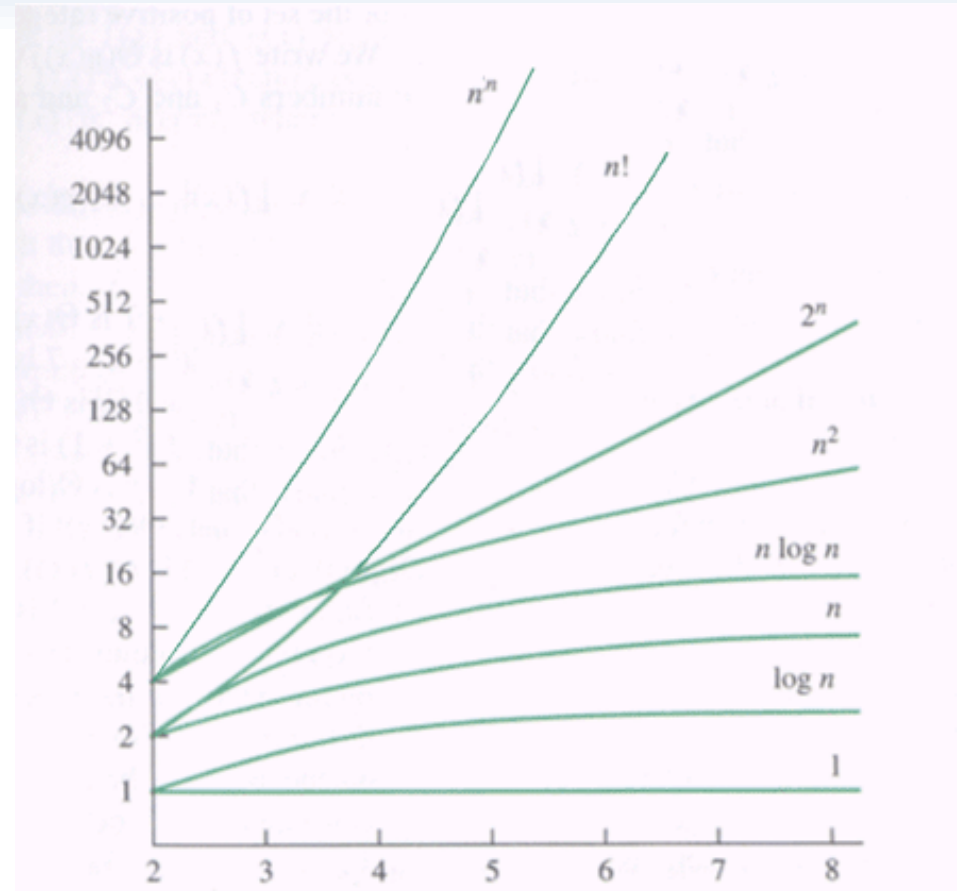
$$T(n) = \log(n)$$



- Estudio asintótico:** al crecer el tamaño del problema el coste computacional del primer algoritmo crece mucho más que el del segundo.
- La complejidad del primer algoritmo parece mayor que la del segundo.

Órdenes de crecimiento

- El **orden de crecimiento** de un algoritmo con complejidad $T(n)$ nos indica, de forma general, cómo varía el coste computacional cuando aumenta el tamaño de la entrada n .
- Puede ser logarítmico, lineal, cuadrático, exponencial, etc.
- Dados dos algoritmos, consideraremos más eficiente (menos complejo) a aquél que tenga un menor orden de crecimiento del coste computacional.



Órdenes de crecimiento

	Logarítmico	Lineal	Casi lineal	Cuadrático	Cúbico	Exponencial	Factorial
Entrada	$\log_2(n)$	n	$n \log_2(n)$	n^2	n^3	2^n	$n!$
10^1	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	$1.0 \cdot 10^3$	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9	$1.1 \cdot 10^{301}$	$4.0 \cdot 10^{2567}$
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}	$2.0 \cdot 10^{3010}$	$2.8 \cdot 10^{35659}$
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}	$1.0 \cdot 10^{30103}$	$2.8 \cdot 10^{456573}$
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}	$9.9 \cdot 10^{301029}$	$8.3 \cdot 10^{5565708}$

Algoritmos Eficientes

Algoritmos Tratables

Algoritmos Intratables

Sólo son prácticos para resolver problemas de pequeño tamaño

Notación Asintótica

- La **notación asintótica** nos permite clasificar las funciones de complejidad de los algoritmos en base a su orden de crecimiento.
- Cada notación permite acotar de diferente manera el orden de crecimiento de un algoritmo.
- Estudiaremos:
 - **Cota superior**: $O(\cdot)$ (Omicron, Big O)
 - **Cota inferior**: $\Omega(\cdot)$ (Omega, Big omega)
 - **Orden exacto**: $\Theta(\cdot)$ (Zeta, Theta)

Notación Asintótica $O(g(n))$

- Definición:

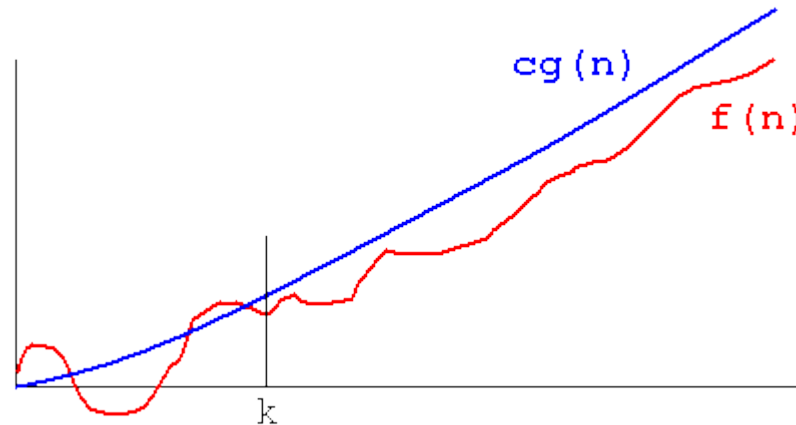
$$O(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c > 0: (\forall n \geq k: f(n) \leq cg(n))\}$$

- Informalmente, $O(g(n))$ es el conjunto de funciones acotadas superiormente por un múltiplo de g .
- Se utiliza para probar que la complejidad de un algoritmo como muy mal se va a comportar como la función g , que se toma como referencia.
- Observa que los valores iniciales de ambas funciones no importan, lo relevante es que a partir de un cierto tamaño de entrada k , f se comporte mejor o igual que cualquier múltiplo de g .

Notación Asintótica $O(g(n))$

- Definición:

$$O(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c > 0: (\forall n \geq k: f(n) \leq cg(n))\}$$



- Ejemplos:

$$\begin{array}{lll} n \in O(n^2) & 100n + 5 \in O(n^2) & 0.5n(n-1) \in O(n^2) \\ n^3 \notin O(n^2) & 0.00001n^3 \notin O(n^2) & n^4 + n + 1 \notin O(n^2) \end{array}$$

Notación Asintótica $\Omega(g(n))$

- Definición:

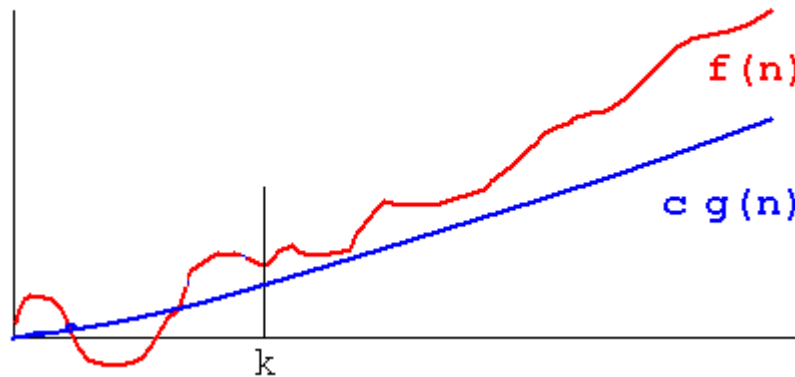
$$\Omega(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c > 0: (\forall n \geq k: f(n) \geq cg(n))\}$$

- Informalmente, $\Omega(g(n))$ es el conjunto de funciones acotadas inferiormente por un múltiplo de g .
- Se utiliza para probar que la función de complejidad de un algoritmo en el mejor caso se va a comportar como la función g , que se toma como referencia.
- Como en el caso de O , los valores iniciales de ambas funciones no importan, lo que es relevante es que a partir de un cierto tamaño de entrada k , f se comporte peor o igual que un múltiplo de g .

Notación Asintótica $\Omega(g(n))$

- Definición:

$$\Omega(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c > 0: (\forall n \geq k: f(n) \geq cg(n))\}$$



- Ejemplos:

$$\begin{array}{lll} n^3 \in \Omega(n^2) & 100n + 5 \in \Omega(n) & 0.5n(n-1) \in \Omega(n^2) \\ n \notin \Omega(n^2) & 0.01n^2 \in \Omega(n) & n^2 + n + 1 \notin \Omega(n^3) \end{array}$$

Notación Asintótica $\Theta(g(n))$

- Definición:

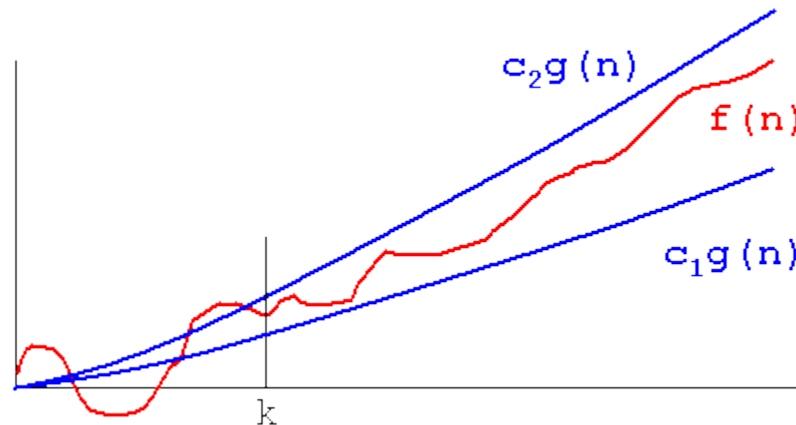
$$\Theta(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c_1 > 0, c_2 > 0: (\forall n \geq k: c_1 g(n) \leq f(n) \leq c_2 g(n))\}$$

- Informalmente, $\Theta(g(n))$ es el conjunto de **funciones** con el **mismo orden** de crecimiento que g .
- Se utiliza para probar que la complejidad de un algoritmo es igual asintóticamente a la función g , que se toma como referencia.
- Como en los dos casos anteriores, **los valores iniciales de ambas funciones no importan**, lo que es relevante es que a partir de un cierto tamaño de entrada k , f crezca de forma **similar** a g .

Notación Asintótica $\Theta(g(n))$

- Definición:

$$\Theta(g(n)) = \{f: \mathbb{N} \rightarrow \mathbb{N} \mid \exists k \geq 0, c_1 > 0, c_2 > 0: (\forall n \geq k: c_1 g(n) \leq f(n) \leq c_2 g(n))\}$$



- Ejemplos:

$$\begin{array}{lll} 100n^2 \in \Theta(n^2) & 100n + 5 \in \Theta(n) & 0.5n(n-1) \in \Theta(n^2) \\ n \notin \Theta(n^2) & 0.01n \notin \Theta(n^2) & n^2 + n + 1 \notin \Theta(n^3) \end{array}$$

Principio de Invarianza

- La complejidad de dos implementaciones de un mismo algoritmo no diferirán más que en una constante multiplicativa.
- Si $T_1(n)$ y $T_2(n)$ determinan el coste temporal de dos implementaciones de un mismo algoritmo, se verifica que:

$$T_1(n) \in \Theta(T_2(n))$$

$$T_2(n) \in \Theta(T_1(n))$$

- Para resolver un problema de una forma más eficiente hay que encontrar un algoritmo mejor, no implementar el mismo algoritmo de otra forma o con otro lenguaje de programación.

Notación Asintótica: propiedades

- **Transitiva** (también para $\Omega(\cdot)$ y $O(\cdot)$):
$$f(n) \in \Theta(g(n)) \quad \text{y} \quad g(n) \in \Theta(h(n)) \Rightarrow f(n) \in \Theta(h(n)).$$
- **Reflexiva** (también para $\Omega(\cdot)$ y $O(\cdot)$): $g(n) \in \Theta(g(n))$
- **Simétrica**: $h(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(h(n))$
- **Simétrica transpuesta**: $h(n) \in \Omega(g(n)) \Leftrightarrow g(n) \in O(h(n)).$
- $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n)) \Leftrightarrow f(n) \in \Theta(g(n)).$

Notación Asintótica: propiedades

- Suma de funciones:

$$\text{Si } f_1(n) \in O(g_1(n)) \text{ y } f_2(n) \in O(g_2(n)), \text{ entonces}$$
$$f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

- Ídem para $\Omega(\cdot)$ y $\Theta(\cdot)$.
- Esta propiedad es útil cuando tenemos algoritmos compuestos de partes diferenciadas que se ejecutan secuencialmente y para las que conocemos la complejidad.
- Ejemplo: Buscar duplicados en un array de n elementos desordenados.
Para ello:
 - Ordenamos el array usando un método de complejidad $O(n^2)$.
 - Comparamos elementos consecutivos, que tiene una complejidad de orden $O(n)$.
 - La complejidad del algoritmo, $T(n)$, está dominada por la primera parte, por lo que, $T(n) \in O(n^2) = O(\max\{n^2, n\})$.

Notación Asintótica: propiedades

- Multiplicación de funciones:

Si $f_1(n) \in O(g_1(n))$ y $f_2(n) \in O(g_2(n))$, entonces
 $f_1(n) \cdot f_2(n) \in O(g_1(n) \cdot g_2(n))$

- Ídem para $\Omega(\cdot)$ y $\Theta(\cdot)$.
- Esta propiedad es útil cuando conocemos que una parte del algoritmo tiene complejidad $O(g_1(n))$ y es invocada $O(g_2(n))$ veces.
- Ejemplo: Ordenación por selección. Para ello buscamos sucesivamente el i -ésimo menor elemento y lo colocamos en la posición i , ($0 \leq i < n - 1$).
 - En cada paso, buscar el menor de $n-i$ elementos es $O(n)$.
 - Hay que repetir la anterior búsqueda un número $O(n)$ de veces.
 - La complejidad del algoritmo es $O(n \cdot n) = O(n^2)$.

Notación Asintótica: propiedades

- Comparación de órdenes de crecimiento:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & f(n) \in O(g(n)) \\ c > 0 & f(n) \in \Theta(g(n)) \\ \infty & f(n) \in \Omega(g(n)) \end{cases}$$

- Ejemplos:

$$\lim_{n \rightarrow \infty} \frac{1}{n^2} = 0 \Rightarrow 1 \in O(n^2) \text{ ó } n^2 \in \Omega(1)$$

$$\lim_{n \rightarrow \infty} \frac{2n}{n} = 2 > 0 \Rightarrow 2n \in \Theta(n) \text{ ó } 2n \in O(n) \text{ ó } 2n \in \Omega(n)$$

$$\lim_{n \rightarrow \infty} \frac{2n^2}{n} = \infty \Rightarrow 2n^2 \in \Omega(n) \text{ ó } n \in O(n^2)$$

Notación Asintótica: propiedades

$$O(\log_a n) = O(\log_b n)$$

Ya que, por la propiedad de cambio de base de los logaritmos

$$\log_a n = \frac{\log_b n}{\log_b a} = \frac{1}{\log_b a} \log_b n = Cte \cdot \log_b n$$

Por esta razón no es necesario especificar la base del logaritmo:
 $O(\log n)$.

Transformaciones matemáticas en límites

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{\infty - \infty}{\infty}$$

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2} \quad \Rightarrow \quad \frac{1}{2}n(n-1) \in \Theta(n^2)$$

- Regla de L'Hôpital:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

donde $f'(n) = d f(n) / dn$

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\log(n)}{\sqrt{n}} &= \frac{\infty}{\infty} \\ \lim_{n \rightarrow \infty} \frac{\log(n)}{n^{1/2}} &= \lim_{n \rightarrow \infty} \frac{1/n}{\frac{1}{2}n^{-1/2}} = \lim_{n \rightarrow \infty} \frac{1}{\frac{1}{2}\sqrt{n}} = \frac{1}{\infty} = 0 \end{aligned}$$

Por tanto, $\log(n) \in O(\sqrt{n})$

Contenido

- Coste computacional de un algoritmo
- Análisis de algoritmos iterativos
- Análisis de algoritmos recursivos
 - Resolución de ecuaciones de recurrencia
- Órdenes de crecimiento y notación asintótica
- El teorema maestro

El Teorema Maestro

- El Teorema Maestro nos da el orden de complejidad del algoritmo sin llegar a conocer la expresión exacta de la función complejidad.
- Se puede aplicar cuando al analizar un algoritmo se obtiene la expresión

$$T(n) = \begin{cases} aT(n/b) + f(n) & n > 1 \\ \Theta(1) & n = 1 \end{cases} \quad \text{con } a \geq 1, b > 1$$

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & \exists \varepsilon > 0 : f(n) \in O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \cdot \log^{k+1} n) & f(n) \in \Theta(n^{\log_b a} \cdot \log^k n) \\ \Theta(f(n)) & \begin{aligned} &\exists \varepsilon > 0 : f(n) \in \Omega(n^{\log_b a + \varepsilon}) \text{ y} \\ &\exists e < 1, n_0 \geq 0 : (\forall n \geq n_0 : a \cdot f(n/b) \leq e \cdot f(n)) \end{aligned} \end{cases}$$

El Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & \boxed{\exists \varepsilon > 0 : f(n) \in O(n^{\log_b a - \varepsilon})} \\ \Theta(n^{\log_b a} \cdot \log^{k+1} n) & f(n) \in \Theta(n^{\log_b a} \cdot \log^k n) \\ \Theta(f(n)) & \begin{array}{l} \exists \varepsilon > 0 : f(n) \in \Omega(n^{\log_b a + \varepsilon}) \text{ y} \\ \exists e < 1, n_0 \geq 0 : (\forall n \geq n_0 : a \cdot f(n/b) \leq e \cdot f(n)) \end{array} \end{cases}$$

- Intuitivamente el teorema maestro está comparando $f(n)$ con $n^{\log_b a}$ y aplicando la propiedad de la suma de funciones.
 - Caso 1: Si $f(n)$ es estrictamente mejor que $n^{\log_b a}$, entonces $T(n) \in \Theta(n^{\log_b a})$.

El Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & \exists \varepsilon > 0 : f(n) \in O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \cdot \log^{k+1} n) & f(n) \in \Theta(n^{\log_b a} \cdot \log^k n) \\ \Theta(f(n)) & \begin{array}{l} \exists \varepsilon > 0 : f(n) \in \Omega(n^{\log_b a + \varepsilon}) \text{ y} \\ \exists e < 1, n_0 \geq 0 : (\forall n \geq n_0 : a \cdot f(n/b) \leq e \cdot f(n)) \end{array} \end{cases}$$

- Intuitivamente el teorema maestro está comparando $f(n)$ con $n^{\log_b a}$ y aplicando la propiedad de la suma de funciones.
 - Caso 1: Si $f(n)$ es estrictamente mejor que $n^{\log_b a}$, entonces $T(n) \in \Theta(n^{\log_b a})$.
 - Caso 2: Si $f(n)$ es del mismo orden que $n^{\log_b a}$, salvo alguna constante logarítmica $\log^k n$, entonces la complejidad es la de ambas funciones aumentando un grado la potencia del factor logarítmico k . $T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$.

El Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & \exists \varepsilon > 0 : f(n) \in O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \cdot \log^{k+1} n) & f(n) \in \Theta(n^{\log_b a} \cdot \log^k n) \\ \Theta(f(n)) & \boxed{\exists \varepsilon > 0 : f(n) \in \Omega(n^{\log_b a + \varepsilon})} \text{ y} \\ & \exists e < 1, n_0 \geq 0 : (\forall n \geq n_0 : a \cdot f(n/b) \leq e \cdot f(n)) \end{cases}$$

- Intuitivamente el teorema maestro está comparando $f(n)$ con $n^{\log_b a}$ y aplicando la propiedad de la suma de funciones.
 - Caso 1: Si $f(n)$ es estrictamente mejor que $n^{\log_b a}$, entonces $T(n) \in \Theta(n^{\log_b a})$.
 - Caso 2: Si $f(n)$ es del mismo orden que $n^{\log_b a}$, salvo alguna constante logarítmica $\log^k n$, entonces la complejidad es la de ambas funciones aumentando un grado la potencia del factor logarítmico k . $T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$.
 - Caso 3: Si $f(n)$ es estrictamente peor que $n^{\log_b a}$ entonces $T(n) \in \Theta(f(n))$. Sólo se puede aplicar si $f(n)$ satisface la condición de regularidad $a f(n/b) \leq e f(n)$ para valores de n lo suficientemente grandes y una constante positiva $e < 1$.

Teorema Maestro. Ejemplos

- $T(n) = 3T(n/4) + n \log n$

Identificamos $f(n) = n \log n$ y $n^{\log_b a} = n^{\log_4 3}$. Como $\log_4 3 < 1$, parece que estamos en el caso 3. Suponemos un $\varepsilon > 0$ cercano a 0.

$$\lim_{n \rightarrow \infty} \frac{n \log n}{n^{\log_4 3 + \varepsilon}} = \lim_{n \rightarrow \infty} \frac{\log n}{n^{\log_4 3 + \varepsilon - 1}} = \lim_{n \rightarrow \infty} \log n \cdot n^{1 - (\log_4 3 + \varepsilon)} = \infty$$
$$\Rightarrow f(n) = n \log n \in \Omega(n^{\log_4 3 + \varepsilon})$$

Por otro lado, hay que encontrar un valor $0 < e < 1$ que satisfaga $af(n/b) \leq ef(n)$

$$3f(n/4) \leq ef(n); 3 \frac{n}{4} \log(n/4) \leq e n \log n;$$

$$\frac{3}{4} n \log n - \frac{3}{4} n \log 4 \leq e n \log n; \frac{3}{4} - \frac{3 \log 4}{4 \log n} \leq e$$

$e = 3/4 < 1$ satisface la inecuación.

Por tanto, $T(n) \in \Theta(n \log n)$

- $T(n) = 2T(n/2) + n \log n$

Identificamos $f(n) = n \log n$ y $n^{\log_b a} = n^{\log_2 2} = n$.

En este caso las dos funciones son del mismo orden, salvo la constante logarítmica $\log^1 n$.

Por tanto, $T(n) \in \Theta(n \cdot \log^{1+1} n) = \Theta(n \cdot \log^2 n)$

Versión reducida del Teorema Maestro

- Caso especial que se puede aplicar cuando $f(n)$ es un polinomio. Es decir, si la función de complejidad tiene la forma $T(n) = aT(n/b) + f(n)$ y $f(n) \in \Theta(n^d)$ con $d \geq 0$.

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & a > b^d \\ \Theta(n^d \log n) & a = b^d \\ \Theta(n^d) & a < b^d \end{cases}$$

```
/**
 * @param l lista con n > 0 elementosr
 * @param min y max, índices máximo y mínimo a suma
 * @return Suma de los elementos de la lista
 * Implementación recursiva
 */
public static double suma(List<Double> l, int min, int max) {
    if (max < min) return 0;
    if (max == min) return l.get(min);
    else {
        double s1 = suma(l, min, (min+max)/2);
        double s2 = suma(l, (min+max)/2+1, max);
        return s1+s2;
    }
}
```

- $T(n) = 2T(n/2) + 1$ y $f(n) \in \Theta(n^0)$.
- Identificamos $a = 2$, $b = 2$ y $d = 0$.
- Como $a = 2 > 2^0 = b^d$ podemos afirmar por el teorema maestro reducido que $T(n) \in \Theta(n) = \Theta(n^{\log_2 2})$